

P2434R0: Nondeterministic pointer provenance

Audience: EWG, SG22

S. Davis Herring <herring@lanl.gov>

November 15, 2022

Introduction

[P2318R1](#) describes a variety of plausible models of pointer provenance that differ principally in how they handle conversions between pointers and integers (including the integer values of the storage bytes for a pointer). (See its §A.4 for discussion of the variants in terms of examples.) It proposes the variant called PNVI-ae-udi, which does seem to provide a good combination of optimization possibilities and support for existing code. However, in several cases it is overly charitable to the programmer: for example, using an explicit copy loop rather than calling `std::memcpy` “exposes” storage, which can interfere with optimization, even if the byte values obviously do not escape. (The paper proposes to eventually add annotations to avoid such unwanted side effects.) It also sometimes depends in an apparently arbitrary fashion on the order of operations with no data dependencies (as in the `pointer_from_integer_1ig.c` example with an exposure of `j` added).

The main alternative is the PVI model, which avoids the notion of storage exposure but imposes further restrictions on integer conversions. These restrictions provide further opportunities for optimization but also complicate the execution model in subtle ways that make it difficult for the programmer to determine whether a manipulation preserves the validity of a pointer (yet to be reconstructed). They also interact badly with serialization of pointers where operations on the converted pointer value are entirely invisible; additional annotations might be required to support this use case.

This paper compares the existing rules for pointers to the proposed provenance models and proposes minor changes to better align them.

Analysis

Pointer values are described in abstract terms (`[basic.compound]/3`); while they “represent the address” of an object, that address may be given a numerical interpretation only via copying the bits into an integer or via the implementation-defined mapping to integers (`[expr.reinterpret.cast]/4`), and there is no specification of how any address is chosen. As such, integers obtained by `memcpying` or casting pointers may be taken to be completely unspecified aside from the (separate) round-trip requirements. (`[basic.align]/1` talks about addresses as ordinal or cardinal numbers of bytes, but nothing else depends on that interpretation.)

This nondeterminism already implies undefined behavior in many of the circumstances that the provenance models are meant to reject. Consider the simple example

```
int main() {  
    int jenny=0;  
    // std::cout << &jenny;  
    *(int*)8675309=1;  
    return jenny;  
}
```

(Assume that the implementation does not specially define a pointer value corresponding to this particular integer.) Even if the address of `jenny` is the suspicious value given, this has undefined behavior under PNVI-ae-udi (because the address of `jenny` is never exposed) and PVI (because the integer literal is not derived from the

address of jenny). However, it is equally undefined in N4917 because there are possible executions of the program ([intro.abstract]/5) where the address is some other value and the cast produces an invalid pointer value. Printing the address is no help: even if the program displays “0x845fed”, that itself can be a manifestation of the undefined behavior.

On the other hand, consider

```
int main() {
    int x,y=0;
    uintptr_t p=(uintptr_t)&x,q=(uintptr_t)&y;
    p^=q;
    q^=p;
    p^=q;
    *(int*)q=*(int*)p;
    return x;
}
```

PVI disallows this manipulation, saying that the values resulting from operations on *p* and *q* do not inherit their provenances; PNVI-ae-udi allows it because it simply observes that both addresses have escaped from the abstract world of pointer values. In the latter case, the compiler would have to support “guessing” addresses even if it could prove that the value of such a pointer doesn’t actually depend on the original pointer. N4917 already handles this case more elegantly: the program has well defined behavior because for any choice of addresses for *x* and *y*, *q* (*p*) ends up being the address of *x* (*y*), so the casts back to pointers produce the swapped pointer values. This interpretation extends to arbitrary integer manipulations and I/O: the operations allowed are precisely those that *reliably* reproduce some address initially obtained, whether via arithmetic, control flow, or data dependencies. In practice it is impossible to detect every pointer value construction that fails to be reliable (that “guesses”), but there is no obligation to do so since the behavior is undefined in such cases.

The “pointer zap” rule ([basic.stc.general]/4) has two purposes: its footnote explains that even examining a pointer into deallocated storage might trap (in the course of loading something like a segment register), but it also serves as a sort of notional “generation counter” for reused parts of the free store. Consider

```
int main() {
    int *p=new int;
    uintptr_t i=(uintptr_t)p;
    delete p;
    p=new int(1);
    if((uintptr_t)p==i) *(int*)i=0;
    return *p;
}
```

PVI rejects this LBYL approach: *i* cannot acquire the provenance of the new *p* merely because it has been compared to another integer with that provenance. (Consider that the comparison might occur in an opaque function.) PNVI-ae-udi allows it because the address of that new object is exposed in the course of making the check; N4917 allows it simply because *i* is cast back to a pointer precisely when it has the same value as the cast of the new *p*. It is only natural that two integers with the same value should have the same behavior.

N4917 does not, however, implement the user-disambiguation (*udi*) provision: in saying that a pointer subjected to a round trip conversion “will have its original value”, [expr.reinterpret.cast]/5 erroneously forbids real implementations that produce the same integer value for pointers to an object and one past the object that immediately precedes it in memory. Similarly, [basic.types.general]/2–3 refer to a singular value for what might be a pointer reconstructed from bytes (and /4 claims that the bits are [sufficient to determine the value](#)). [bit.cast]/2 acknowledges the possibility of more than one value with the same value representation, but leaves it unspecified which is produced.

Proposal

To implement *udi*, apply the same *angelic nondeterminism* by which [implicit object creation](#) selects the objects to create: if any pointer value exists that corresponds to the integer and gives the program defined behavior, one such value is the result. This change affects [basic.types.general]/2–4, [expr.reinterpret.cast]/5, and [bit.cast]/2 (which currently instead plays into the general *demonic nondeterminism* of [intro.abstract]/5). This paper does not attempt to address the situation of modifying a pointer by [storing to part of its object representation](#), but it changes the simple memcpy case to avoid eventually giving different behavior to direct and circuitous means of accomplishing a bit-wise copy.

To avoid confusing inconsistencies with comparing their integer representations (on implementations where each address has just one such), restrict [expr.eq] to provide consistent results for any pair of pointer values. This change cannot be detected during constant evaluation ([expr.const]/5.24); following P2318R1, we could consider actually comparing the addresses other than during constant evaluation.

Wording

Relative to N4917.

#[basic.types]

#[basic.types.general]

Move paragraphs 2 and 3 to [basic.types.trivial] (*q.v.*).

Change paragraph 4:

- The *object representation* of an object of type τ is the sequence of N unsigned char objects taken up by the object of type τ , where N equals `sizeof( $\tau$ )`. The *value representation* of an object of type τ is the set of bits that participate in representing a value of type τ . Bits in the object representation that are not part of the value representation are *padding bits*. ~~For trivially copyable types, the value representation is a set of bits in the object representation that determines a value, which is one discrete element of an implementation-defined set of values.~~ [Footnote: [...] — end footnote]

#[basic.types.trivial]

Add this subclause before [basic.fundamental]:

- Each trivially copyable type τ has an implementation-defined set of discrete values. A *bit value* is a member of an implementation-defined disjoint partition of the set of values; for scalar types other than object pointer types, each contains no more than one value. The value representation of an object of type τ determines a bit value for that object. When an object *acquires* a bit value, its value becomes an unspecified member of that bit value that would result in the program having defined behavior, if any.

Move paragraphs 2 and 3 here from [basic.types.general] and change them:

- ~~For any~~ **If an** object ~~(other than of type τ is not~~ a potentially-overlapping subobject) ~~of trivially copyable type τ ,~~ whether or not the object holds a valid value of type τ , the underlying bytes ([intro.memory]) making up the object can be copied into an array of char, unsigned char, or `std::byte` ([cstddef.syn]). [Footnote: [...] — end footnote] If the content of that array is copied back into the object, the object ~~shall subsequently hold~~ **acquires** its original **bit** value.

[Example:

[...]

— end example]

- For two distinct **such** objects obj1 and obj2 ~~of trivially copyable type T, where neither obj1 nor obj2 is a potentially overlapping subobject,~~ if the underlying bytes ([intro.memory]) making up obj1 are copied into obj2, *[Footnote: [...] — end footnote]* obj2 ~~shall subsequently hold~~ **acquires** the ~~same bit~~ value **as of** obj1.

[Example:

[...]

— end example]

#[expr]

#[expr.reinterpret.cast]

Change paragraphs 4 and 5:

- A pointer can be explicitly converted to any integral type large enough to ~~hold~~ **distinguish** all **bit** values of its type. The mapping function is implementation-defined.

[Note: It is intended to be unsurprising to those who know the addressing structure of the underlying machine. — end note]

[...]

- A value of integral type or enumeration type can be explicitly converted to a pointer. ~~A pointer converted to an integer of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value; mappings between pointers and integers are~~ **If the value is equal to that produced by converting one or more pointer values to an integral type, the result is an unspecified choice among those values that would result in the program having defined behavior. If no such value exists, the behavior is undefined.** ~~Otherwise, the result is~~ implementation-defined.

[Note: It can be an invalid pointer value. — end note]

#[expr.eq]

Insert before paragraph 3:

- Any two pointer values or two pointer-to-member values either compare equal or compare unequal.

[Note: Repeated comparisons are consistent so long as neither value is an invalid pointer value. — end note]

Change paragraph 6:

- If two operands compare equal, the result is true for the == operator and false for the != operator. If two operands compare unequal, the result is false for the == operator and true for the != operator. ~~Otherwise, the result of each of the operators is unspecified.~~

#[bit.cast]

Change paragraph 2:

- *Returns*: An object of type `To`. Implicitly creates objects nested within the result (`[intro.object]`). Each bit of the value representation of the result is equal to the corresponding bit in the object representation of `from`. Padding bits of the result are unspecified. ~~For `t`~~^{The result and each object created within it, if there is no value of the object's type} ^{acquire the bit values} corresponding to the value representation produced; ^{if any such bit value is empty,} the behavior is undefined. ~~If there are multiple such values, which value is produced is unspecified.~~ A bit in the value representation of the result is indeterminate if it does not correspond to a bit in the value representation of `from` or corresponds to a bit of an object that is not within its lifetime or has an indeterminate value (`[basic.indet]`). For each bit in the value representation of the result that is indeterminate, the smallest object containing that bit has an indeterminate value; the behavior is undefined unless that object is of unsigned ordinary character type or `std::byte` type. The result does not otherwise contain any indeterminate values.

Acknowledgments

Thanks to Richard Smith and Peter Sewell for reviewing an early, overcomplicated draft of this paper.